DAY 2

Task 1 — Preprocessing Plan & Implementation

Preprocessing Plan

The Adult Income dataset contains both numerical and categorical features, so separate preprocessing pipelines were used.

Numerical Features:

Age, Fnlwgt, Education-num, Capital-gain, Capital-loss, Hours-per-week.

Categorical Features:

Workclass, Education, Marital-status, Occupation, Relationship, Race, Sex, Native-country.

Missing values represented by ? were replaced with NaN.

Preprocessing Methods

For numerical features, median imputation followed by StandardScaler was used. Median imputation was selected because it is more robust to outliers and skewed data than mean imputation. StandardScaler was used to bring numerical features to a comparable scale.

For categorical features, most-frequent imputation followed by OneHotEncoder was used. One-Hot Encoding was selected because categorical features do not have a natural numerical order. handle_unknown="ignore" was used to prevent errors from unseen categories in development or test data.

A ColumnTransformer was used to combine both pipelines into one reproducible preprocessing workflow.

Alternatives Considered

Mean imputation, KNN imputation, and iterative imputation were considered but skipped because they are either more sensitive to outliers or more complex. Ordinal encoding was skipped because it can create an incorrect order between categories, while target encoding was avoided because of potential data leakage.

Data Leakage Prevention

The preprocessing pipeline was fitted **only on the training data**. The development and hold-out test sets were only transformed using the fitted pipeline. This prevents information from the evaluation data from influencing preprocessing.

Conclusion

The preprocessing pipeline successfully prepares both numerical and categorical features for supervised learning. Median imputation, StandardScaler, most-frequent imputation, and One-Hot Encoding provide a simple and reliable baseline while maintaining a leakage-free and reusable workflow.

Task 2 — Train Two Supervised Models

Model Implementation

Two supervised machine learning models were implemented using Scikit-learn pipelines: Logistic Regression and Decision Tree Classifier. Both models were combined with the preprocessing pipeline created in Task 1 so that numerical and categorical features were processed consistently.

Logistic Regression

Logistic Regression was used as a simple and interpretable baseline classification model. The liblinear solver was selected because it supports L2 regularization. A fixed random_state=42 was used to ensure reproducible results.

Decision Tree Classifier

A Decision Tree Classifier was implemented to capture non-linear relationships and feature interactions in the data. A fixed random_state=42 was used to make the results reproducible.

Training

Both models were trained using **only the training dataset**. The hold-out test set was not used during model training or preprocessing fitting. This ensures that the final test evaluation remains unbiased and prevents data leakage.

Conclusion

Both Logistic Regression and Decision Tree models were successfully trained using complete preprocessing pipelines. Logistic Regression provides an interpretable linear approach, while the Decision Tree can capture non-linear patterns and interactions.

Task 3: Model Evaluation on Hold-Out Test

- Evaluated Logistic Regression and Decision Tree on the unseen hold-out test set.
- Calculated Accuracy, Precision, Recall, F1, ROC AUC, and PR AUC.
- Compared both models with Day 1 baseline models.
- Generated confusion matrices to analyze False Positives and False Negatives.
- Plotted ROC curves and Precision–Recall curves for both models.
- F1, ROC AUC, and PR AUC were considered important because the dataset is imbalanced.
- Overall, both supervised models performed better than the simple baselines.
- Logistic Regression offers better interpretability, while Decision Tree captures nonlinear relationships.
- Final model selection will depend on metrics and FP/FN trade-offs.

Task 4: Interpretability Check

- Logistic Regression: Coefficients were analyzed after preprocessing and one-hot encoding.
- get_feature_names_out() was used to map coefficients to feature names.
- Top 10 positive and 10 negative coefficients were identified.
- Positive coefficients → higher probability of >50K income.
- Negative coefficients → higher probability of ≤50K income.
- Decision Tree: Tree depth, number of leaves, and train/test accuracy were checked.
- Train-test accuracy gap was used to check overfitting.
- Top 3 tree splits were identified to understand important decision features.
- Logistic Regression is more interpretable, while Decision Tree captures nonlinear relationships.

Task 5: Write-Up & Model Selection

- Compared Logistic Regression and Decision Tree using hold-out test metrics.
- Evaluated Accuracy, Precision, Recall, F1, ROC AUC, and PR AUC.
- Logistic Regression was considered a strong candidate because it is accurate and easy to interpret.
- Decision Tree can capture nonlinear relationships and feature interactions.
- Decision Tree was checked for overfitting using training vs. test performance.
- Model selection will consider F1, ROC AUC, PR AUC, and False Positive/False Negative trade-offs.
- The same preprocessing pipeline will be reused to avoid data leakage.
- Tomorrow, we will test different imputation methods, regularization, tree depth, class weighting, and probability thresholds.